

Misleading Identifiers Change What Code Traces Write

Anonymous ACL submission

Abstract

A misleading identifier can change the intermediate value a code trace writes. We test short Python functions whose matched versions differ in one identifier, across seven instruction-tuned models and three demonstration sets. In a terse trace, a variable assigned `len(xs)` but named like a sum produces +12.4 to +58.4 points of written-lure excess in three models. Patching the identifier at an early layer or the writing position at a later layer removes most lure writes in the tested pairs. Including the expression beside the value in demonstrations sharply reduces the effect. The effect is smaller in REPL and comment formats, weaker at 34B parameters, and undetected on renamed CRUXEval functions in direct-answer and prose regimes. These results establish a format-dependent naming error.

1 Introduction

A variable named `sum_all` is assigned `len(xs)`. On input `[5, 3]`, the code gives it value 2, but its name suggests 8. In Figure 1, OLMo-2-7B writes 8 when its examples show a terse trace such as `v = 3`. It writes the correct value when the same examples include the expression: `v = len(xs) = 3`. The program and input are unchanged.

Misleading identifiers reduce accuracy on code tasks, and step-by-step reasoning can leave that loss unresolved (Le et al., 2025; Lam et al., 2025; Le et al., 2026). A wrong trace does not tell us whether the model lacks the correct value or fails to write an available one. Kudo et al. (2026) study intermediate values during arithmetic chains with probes and patching. We use these tools on code, where an identifier can suggest an executable operation that differs from its assignment.

We compare matched functions with congruent, neutral and misleading names. Executing the implied operation supplies the name’s *lure value*;

executing the code supplies the true value. The strongest effect occurs when a length-computed variable has a sum-like name. Patches at the name and writing position reduce lure writes, as does including the expression in the examples. Probes compare the computed digit with the value the trace writes.

2 Study design

Each problem is a short Python function over a list of distinct digits. Assignments apply list aggregates (`len`, `sum`, `max`, `min`) or integer operations, followed by a return and a call. True intermediate and final values are single digits. The main level has three dependent assignments; Appendix A gives the full design. We test seven instruction-tuned models with 1,000 matched problems per level and target, with three fixed sets of three worked examples. CodeLlama-34B supplies an additional scale check.

Each matched pair differs only in one target identifier. A congruent name implies the assigned operation; a misleading name implies another. Its lure value must differ from every value, constant and input element in the problem. This prevents copying from explaining a lure write and excludes `max` and `min` as lure operations on the aggregate step. We also check which operation each model associates with each identifier. This check is diagnostic: names were retained even when they failed the planned meaning gate (Appendix B).

In the *direct* regime, models write the return value. In *trace*, they first write each assignment’s value, as in `qq = 3, v = 6, vv = 9`. The *trace-expr* comparison adds the expression to each demonstrated step, as in `qq = len(xs) = 3`. REPL, comment and free-prose formats test the effect’s scope (Appendix D).

Our main measure is *written-lure excess*: the rate at which a misleading trace writes the lure

(a) conflicting values

```
def f(xs):  
    sum_all = len(xs)  
    ww = sum_all + 3  
    qq = ww + 4  
    return qq  
f([5, 3])
```

code: `len(xs) = 2`

name: `sum(xs) = 8`

the call returns 9

(b) OLMo-2-7B's traces

Examples show: `v = 3`

`sum_all = 8, ww = 11, qq = 15`

answers 15 (wrong)

Examples show: `v = len(xs) = 3`

`sum_all = len(xs) = 2, ww =`

`sum_all + 3 = 5, qq = ww + 4 = 9`

answers 9 (correct)

*Same problem and worked examples.
Only the examples' format changes.*

Figure 1: **(a)** The code and the name imply different values for one variable. **(b)** OLMo-2-7B's trace for the same problem under two demonstration formats: values alone (red) or expressions with values (green). Table 2 gives the aggregate effects.

at the target step, minus the neutral twin's rate of writing that same digit. We measure lure answers similarly in the direct regime. We call an effect reliable when its sign agrees across all three demonstration sets and a 95% cluster-bootstrap interval excludes zero. To support a small-effect claim, a 90% interval must fit inside the preregistered ± 2 -point margin. Other cases remain inconclusive (Appendix G).

For three models, a linear probe predicts the target digit from a hidden state at one token and layer. Probes train on neutral traces, with three training seeds. We read the *decision token*: the = immediately before the first traced value. We supply the correct trace prefix up to this token, before the model writes a value. Patching replaces either the name tokens or this position with a matched neutral twin's activations at one layer. A neutral-for-neutral patch measures disruption. Probe controls and implementation details are in Appendix E. Probe training and evaluation use disjoint program computations.

3 Results

3.1 Terse traces follow a misleading name

Table 1 shows the main level. Under sum-like names, length-computed variables produce +58.4,

+43.4 and +12.4 points of written-lure excess in OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B. On neutral programs, their trace accuracies are 95.7, 97.4 and 99.6%, respectively. These models usually trace neutral programs correctly.

The sum is not these models' usual error on a length step. On wrong neutral traces, sum writes account for only 14–32% of errors; maximum values and off-by-one lengths are more common. The sum-like name changes which wrong value they write. CodeLlama-7B's larger effect occurs on a maximum-computed variable (+10.7 points); CodeGemma-7B shows that pairing at level 3. The full pairing matrix is in Appendix C. Direct-answer lure effects are much smaller: -0.2 to +3.1 points at this level.

3.2 Readouts and patches at the writing position

At the decision token, neutral-trained probes recover the computed digit with accuracy of 85%, 90% and 100% in the three affected models. Each layer is selected by neutral accuracy, averaging three probe seeds. Training uses separate programs; renamed test computations are excluded. The name-identity controls score 1.00, 1.00 and 0.80 on neutral programs. These readouts measure decodable information, not whether the model

Model	Trace acc.	Direct: lure	Trace: len→sum	Trace: max→sum
OLMo-2-7B	95.7	+0.4	+58.4* [+53.0, +63.5]	+1.9 [+1.1, +3.0]
Llama-3.2-3B	97.4	+0.8	+43.4* [+38.5, +48.1]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	99.6	+0.5	+12.4* [+8.8, +16.0]	+0.0 [+0.0, +0.0]
CodeLlama-7B	91.7	+1.0	+3.7* [+2.2, +5.4]	+10.7* [+7.9, +13.7]
CodeGemma-7B	99.2	+1.5*	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]
Gemma-3-4B	99.1	+0.8	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
OLMo-2-1B	37.8	+3.1*	+3.7* [+2.2, +5.5]	+4.2* [+2.6, +5.9]
CodeLlama-34B	98.9	–	+0.0 [+0.0, +0.0]	+1.8* [+0.7, +3.1]

Table 1: Level 5, means over three demonstration sets. *Trace acc.*: neutral accuracy. *Direct: lure*: excess lure answers for the queried variable. The trace columns give written-lure excess for length- or maximum-computed variables with sum-like names, with 95% bootstrap intervals. Effects are percentage points. * marks a consistent sign across sets and an interval excluding zero.

Model	trace	REPL	comment	trace-expr
OLMo-2-7B	+58.4*	+9.5*	+0.0	+3.4*
Llama-3.2-3B	+43.4*	+12.6*	+6.8*	+0.4
Llama-3.1-8B	+12.4*	+0.2	+0.1	+0.0

Table 2: Written-lure excess for length-computed variables with sum-like names under four demonstration formats. Programs and example sets are fixed; effects are percentage points. Intervals appear in Table 6.

uses the digit to generate its trace. Appendix E gives the demonstration-set repeats.

Patching independently tests sensitivity to the name and the writing position (Figure 2). Replacing name tokens at layer 0 removes 97% and 100% of lure writes in the two models with aligned twins. Name-site removal declines in later layers. At the decision token, removal is weak early and strong from the middle of the network onward; all-layer replacement removes 97–100%. The neutral-word control changes at most 1.0% of outputs. The patches identify positions sensitive to the misleading name. They replace an entire hidden-state vector, so they cannot identify which feature changes the output.

3.3 Showing the expression reduces the effect

We keep the programs, models and demonstration sets fixed and change the examples from $v = 3$ to $v = \text{len}(xs) = 3$. Written-lure rates fall by 55, 43 and 12 points in the three models (Table 2). OLMo-2-7B retains +3.4 points of excess; the two Llama effects approach zero. Thus the expression format sharply reduces the error without eliminating it in every model.

REPL transcripts also omit expressions, yet their effects are several times smaller; comment traces reduce them further. The larger checkpoint has a smaller maximum-step effect: CodeLlama-

34B has +0.0 excess on length steps and +1.8 [+0.7, +3.1] on maximum steps. The latter interval crosses the equivalence margin. Renamed CRUXEval functions show no detectable accuracy effect in direct-answer or prose regimes; those tests do not include the synthetic trace format (Appendix F).

4 Related work

Misleading identifiers affect real-code accuracy (Le et al., 2025; Lam et al., 2025; Le et al., 2026; Orvalho and Kwiatkowska, 2025; Gao et al., 2023). Our matched synthetic tasks make the implied operation executable and locate where its value can enter a trace. Kudo et al. (2026) study arithmetic value readouts, while Anonymous (2026) relate reduced lure answers to decodable values in a companion arithmetic study. Here misleading names change trace writes. The two studies do not establish a shared mechanism. Prompt and chain perturbations test faithfulness externally (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025); we intervene at the value-writing position. Earlier Stroop studies focus on next-token prediction (Stroop, 1935; Geirhos et al., 2019; Wang, 2026; Hu et al., 2026).

5 Conclusion

In the tested terse traces, a sum-like identifier can change the value the model writes. Name-site and decision-site patches reduce these lure writes, as does including the expression in demonstrations. The effect’s dependence on format, operation and model limits its use as an account of errors on natural code.

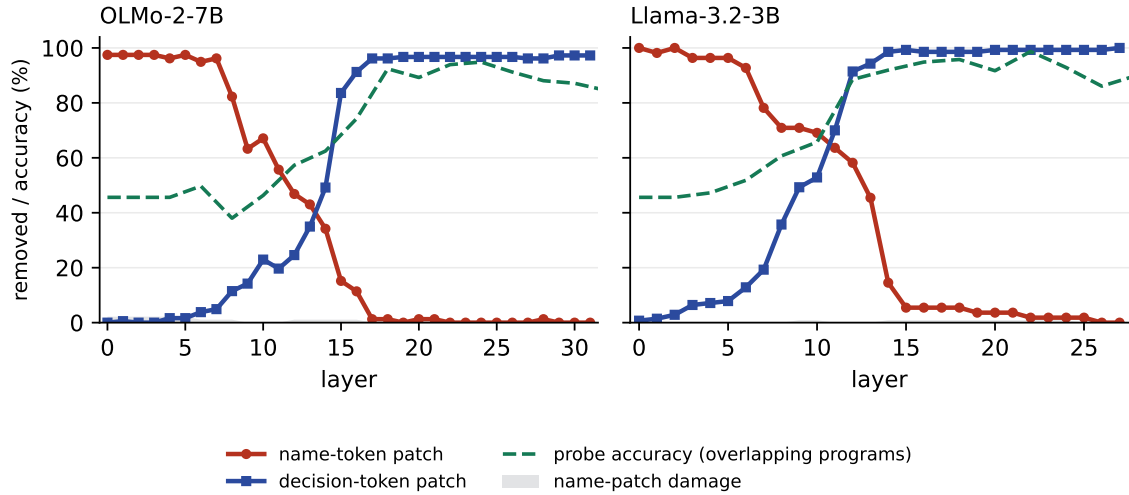


Figure 2: Patching removes lure writes at different layers depending on the site: name tokens (red) or decision token (blue). Dashed green: true-value probe accuracy on programs excluded from training. Grey: damage to correct predictions from the name patch. Llama-3.1-8B’s affected identifier, `sum_all`, lacks a token-aligned neutral twin, so its name-site curve is unavailable.

Limitations

The programs are short and synthetic, with single-digit true values. The format with the largest effect is constructed for this study; REPL and comment formats show less interference. The natural-code check covers 89 renamed CRUXEval functions in direct-answer and prose regimes. Its intervals bound accuracy differences in those settings, not lure writes in synthetic traces.

We study models up to 34B parameters, with internal measurements on three smaller models. Name-identity controls measure identifier information in the readout. A readable digit does not establish that the model uses it during generation. Layers are selected on neutral evaluation accuracy, so those neutral scores are descriptive. Patches replace mixed representations and cannot exclude side effects. A token-aligned name patch is unavailable for Llama-3.1-8B’s `sum_all` cases. The registered REPL prediction was not supported, and identifiers were retained despite failures of the planned meaning gate (Appendices D and B). The model panel also excludes models of Chinese origin and therefore omits some strong open code models tested by Le et al. (2026).

References

- Anonymous. 2026. Chain of thought limits lure answers in arithmetic. Under review.
- Yanda Chen et al. 2025. Reasoning models don’t

always say what they think. *arXiv preprint arXiv:2505.05410*.

Shuzheng Gao, Cuiyun Gao, Chaozheng Wang, Jun Sun, David Lo, and Yue Yu. 2023. Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1933–1945. ArXiv:2207.11104.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.

Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiušė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*.

Pedro Orvalho and Marta Kwiatkowska. 2025. Are large language models robust in understanding code against semantics-preserving mutations? *arXiv preprint arXiv:2505.10443*.

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Task levels and identifiers

Levels vary dependency depth, distractors and which assignment receives the misleading name. The primary comparison uses level 5. A function is generated once; renaming preserves its statements, input and executable result. Table 4 lists the operation suggested by each identifier. Sum and length are admissible lure operations on the aggregate step; minimum and maximum are excluded because their result is already present in the input.

B Identifier meanings

For each identifier and model, we compare log-probabilities of candidate right-hand sides after a prefix such as `def f(xs):\n total =`. The highest-scoring operation records what that identifier evokes. Of 25 names, 12 match the intended operation in at least 80% of models. Most sum-family names instead evoke length in 2 of seven models. This check matters when interpreting a small effect: an identifier need not suggest the same operation to every model. The design planned to retain only names passing the 80% panel gate. The runs instead retained the full name table, so the reported comparisons depart from that selection rule. The results concern these named groups; they do not establish that every model reads each name as intended.

C Operation and name pairings

Table 5 reports all tested aggregate/name pairings. The reverse of the main length-to-sum conflict is weak at level 5: sum-computed variables with length-like names have excess within two points for models above the neutral-accuracy floor. At level 3, OLMo-2-7B has +4.4 points of excess in that pairing. Gemma-3-4B shows no main-cell effect, consistent with its identifier-meaning check. OLMo-2-1B’s neutral accuracy is only 37.8%, making its small, broadly distributed effects harder to interpret as a specific trace failure.

D Demonstration formats

All few-shot comparisons hold programs, demonstration identities and model weights fixed. The trace writes `qq = 3`; trace-expr writes `qq = len(xs) = 3`; REPL writes `>>> qq` followed by 3; comment mode annotates each assignment with its value. Prose asks for step-by-step reasoning without demonstrations. Table 6 gives the paired excess and interval for each few-shot format.

On the main conflict, REPL excess is +9.5 and +12.6 points for OLMo-2-7B and Llama-3.2-3B, compared with +0.2 for Llama-3.1-8B. Comments leave +6.8 points in Llama-3.2-3B and near-zero excess in the other two. Prose has little initial lure writing, so asking it to state or recompute values does not test removal of the large trace effect. The prediction that REPL would stay within a factor of two of trace was not supported; the comment prediction held only for two models.

Level	Neutral program (body)	Traced steps
1	<code>v = len(xs); return v</code>	1
2	<code>v = len(xs); w = v * 2; return w</code>	2
3	<code>v = len(xs); w = max(xs); u = v + w; return u</code>	3
4	level 3 with an unused fourth assignment	3
5	<code>v = len(xs); w = v * 2; u = w + 3; return u</code>	3
6	level 5 with the misleading name on the middle step	3

Table 3: The levels, one example each. The aggregate in the first statement is any of len, sum, max, min; unary operations are $\ast$, $//$, $+$, $-$. The misleading name sits on the first statement at levels 1–5 and on the second at level 6.

Family	Names	Implied operation
len	count, n_items, length	<code>len(xs)</code>
sum	total, sum_all, acc	<code>sum(xs)</code>
max	largest, max_val, peak	<code>max(xs)</code>
min	smallest, min_val, low	<code>min(xs)</code>
double	double, twice	<code>2 * a</code>
half	half	<code>a // 2</code>
succ	succ, next_val	<code>a + 1</code>
pred	pred, prev_val	<code>a - 1</code>
add	combined, both	<code>a + b</code>
sub	diff, gap	<code>a - b</code>
mul	product, scaled	<code>a * b</code>

Table 4: The lure table. The gate (Appendix B) scores each name by the log-probability each model assigns to the candidate right-hand sides; 12 of 25 names are read as intended by at least 80% of models. The min and max names cannot be lures on the aggregate step (their value is an input element) and are used as congruent and alternative-lure names.

The paired trace-minus-trace-expr lure-rate reductions are 55 points [49.9, 60.2], 43 [38.2, 47.7] and 12 [8.8, 16.0] for OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B. These 95% intervals keep each matched program’s demonstration-seed repeats together.

E Probe and patching details

We follow Kudo et al.’s 10-class linear-probe recipe, with three seeds and neutral-only training. Each probe predicts a target digit from one residual-stream position and layer. Name-identity controls use deterministic random labels keyed on the identifier (Hewitt and Liang, 2019). Neutral names and values vary independently, and misleading names are absent from training. The original implementation reused test programs in training. The reported reruns replace those scores: they train on 2,000 independent neutral programs and evaluate 285 misleading-name programs, with overlap guards that ignore name spelling. Table 7 reports the name controls alongside digit accuracy.

The main scores use demonstration set 7. Repeating training and evaluation with sets 11 and 13 gives true-digit accuracies of 90–90%, 90–90% and 90–90% across the three sets. Each repeat

uses the same independent training programs and three probe seeds; its layer is selected by that set’s neutral accuracy. These repeats assess prompt sensitivity, not a causal role for the decoded digit. Table 7 gives both conditions’ readouts and controls.

We supply the correct trace token by token when caching probe inputs. The decision position is the = before the first traced value; later values cannot affect this position under causal attention. Patching uses only the prompt and this trace prefix. Name patches replace every aligned occurrence of the identifier; decision patches replace only that writing position. Both use one layer at a time and include a neutral-for-neutral disruption control (Zhang and Nanda, 2024). The alternative-lure control is uninformative in the main cell: every sum-family identifier implies the same sum, so there is no distinct source lure. Llama-3.1-8B’s affected two-token name lacks aligned neutral twins for the name-site intervention.

F Scope checks

Operation and scale. Moving the misleading name to a unary middle step produces no reliable effect for models above the accuracy floor; the largest excess is +0.6 points. CodeLlama-34B’s

Model	len→sum	max→sum	min→sum	sum→len	max→len
OLMo-2-7B	+17.6* / +58.4*	+0.9 / +1.9	+0.0 / +0.2	+4.4* / +0.0	+0.0 / +0.0
Llama-3.2-3B	+23.6* / +43.4*	+1.1 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Llama-3.1-8B	+1.5 / +12.4*	+0.7 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
CodeLlama-7B	+0.9 / +3.7*	+7.5 / +10.7*	+1.0 / +1.7	+1.5 / +0.0	+0.9 / +0.6
CodeGemma-7B	+0.0 / +0.1	+10.8* / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Gemma-3-4B	+0.0 / +0.0	+0.2 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
OLMo-2-1B	+1.2 / +3.7*	+2.8* / +4.2*	+0.3 / -0.2	+10.6* / +11.1*	+9.6* / +5.6*

Table 5: Written-lure excess at the aggregate step, trace regime, level 3 / level 5, three sets of worked examples pooled; * as in Table 1.

Model	trace	REPL	comment	trace-expr
OLMo-2-7B	+58.4* [+53.0, +63.5]	+9.5* [+6.9, +12.0]	+0.0 [+0.0, +0.0]	+3.4* [+2.0, +5.1]
Llama-3.2-3B	+43.4* [+38.5, +48.1]	+12.6* [+9.4, +15.9]	+6.8* [+4.4, +9.2]	+0.4 [+0.0, +0.9]
Llama-3.1-8B	+12.4* [+8.8, +16.0]	+0.2 [+0.0, +0.6]	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]

Table 6: Table 2 with 95% bootstrap intervals over matched problems.

length-to-sum effect is +0.0, while its maximum-to-sum effect is +1.8 points [+0.7, +3.1]. The latter is a reliable positive effect, but its interval extends beyond the two-point margin. Its small point estimate cannot be reported as equivalence. The larger-checkpoint result supports the registered maximum-to-sum direction and reliable-effect criterion, while its magnitude is much smaller than CodeLlama-7B’s.

Natural code. We select 89 CRUXEval functions with length, loop-counter or maximum assignments, rename the target by AST rewriting to v or total, and verify execution after renaming. Direct and prose accuracy contrasts all have intervals covering zero on the 84 functions whose renamed target is used downstream. Their end-points stay within 10 and 13 points, respectively (Table 9). These are bounded results for those functions and prompts; they do not test the terse few-shot trace used in the synthetic study.

G Effect and equivalence checks

Each contrast subtracts the neutral twin’s rate for the same digit. We resample matched problems with demonstration-seed replicates together and use 95% cluster-bootstrap intervals. Reliability also requires a consistent sign across all three demonstration sets.

Equivalence applies the ± 2 -point margin to 90% intervals from the same cluster bootstrap (two one-sided 5% tests), using 2,000 draws. Across 192 tested contrasts, 150 intervals lie wholly inside the margin, 36 contrasts have reliable effects without meeting that bound, and 6

meet neither criterion. A small estimate or an interval covering zero does not by itself establish equivalence.

Model	Set	Layer	Neutral	Control	Misleading	Control	Majority
OLMo-2-7B	7	20	90.0	80.0	90.0	10.0	30.0
OLMo-2-7B	11	20	90.0	80.0	90.0	10.0	30.0
OLMo-2-7B	13	20	90.0	80.0	90.0	10.0	30.0
Llama-3.2-3B	7	20	90.0	80.0	90.0	10.0	30.0
Llama-3.2-3B	11	20	90.0	80.0	90.0	10.0	30.0
Llama-3.2-3B	13	20	90.0	80.0	90.0	10.0	30.0
Llama-3.1-8B	7	20	90.0	80.0	90.0	10.0	30.0
Llama-3.1-8B	11	20	90.0	80.0	90.0	10.0	30.0
Llama-3.1-8B	13	20	90.0	80.0	90.0	10.0	30.0

Table 7: Disjoint code probes, three demonstration sets. Neutral and misleading are true-digit accuracy; each control column gives name-identity accuracy for that condition. Majority predicts the most frequent digit in training on the misleading test programs. Rates are percentages, averaged over three probe seeds. Layers are selected by neutral accuracy.

Model	Name tokens			Decision token			
	L0	damage	<10% by	50% at	90% at	all layers	damage
OLMo-2-7B	97%	2%	L17	L15	L16	97%	2%
Llama-3.2-3B	100%	0%	L15	L10	L12	100%	0%
Llama-3.1-8B	—	—	—	L8	L9	100%	0%

Table 8: Layers behind Figure 2. Name tokens: lure writes removed by replacing every token of the name at layer 0, damage to correct instances, and the first layer after which removal falls below 10%. Decision token: first layer at which a single-layer replacement removes 50% and 90% of lure writes, removal with every layer replaced, and its damage. Llama-3.1-8B’s lure writes fall on a two-token name without a token-aligned twin.

Model	Direct acc.	Direct Δ	Prose acc.	Prose Δ
OLMo-2-7B	10	+1.2 [+0.0, +3.6]	22	-1.2 [-9.5, +8.3]
Llama-3.2-3B	19	+1.2 [-2.4, +4.8]	25	+3.6 [-3.6, +10.7]
Llama-3.1-8B	22	-3.6 [-9.5, +2.4]	40	+0.0 [-9.5, +9.5]
CodeLlama-7B	18	+2.4 [+0.0, +6.0]	13	+2.4 [-4.8, +9.5]
CodeGemma-7B	18	+0.0 [-4.8, +4.8]	22	+2.4 [-8.3, +13.1]
Gemma-3-4B	15	-2.4 [-6.0, +0.0]	45	+4.8 [-1.2, +11.9]
OLMo-2-1B	16	-3.6 [-8.3, +0.0]	3	+6.0 [+0.0, +11.9]

Table 9: 89 CRUXEval functions that assign a variable from `len(. . .)` (48), a loop counter (38) or `max(. . .)` (3), of which 84 use it downstream; each rendered with the original name, the neutral name `v`, and the misleading name `total`. Accuracy on all neutral renderings, and the paired misleading-minus-neutral difference in points on the downstream-used subset, with a bootstrap interval over functions.